

Absolutely. I'll post the complete Markdown here so you can copy it directly.

YUNITRADE Inventory System — Supabase-Based Concept Plan

Purpose: Working sketch/concept for rebuilding the current YUNITRADE Withdrawal System into a shared desktop + mobile inventory system using Supabase as the backend.

Source basis: The existing `index.html` and `Code.gs`. Existing terminology and workflows are preserved where supported by the current system. Where the existing source does not fully define a business rule, it is marked as an open question rather than assumed.

1. Current System — What the New System Must Preserve

The current application is a browser-based **YUNITRADE Withdrawal System** using an HTML/JavaScript frontend connected to Google Sheets through Google Apps Script.

The current system includes workflows/features such as:

- PIN-based login
- USER and ADMIN roles
- ADMIN-only delete functionality
- Stocks / inventory
- Withdrawal records
- Received records
- Beginning Inventory
- Sales Orders
- Served records
- 3M Yards
- Partial Rolls
- Search
- Item-code autocomplete
- Roll splitting
- Synchronization indicators
- Audit/reference information
- Soft-delete behavior for some records
- Local/pending synchronization behavior

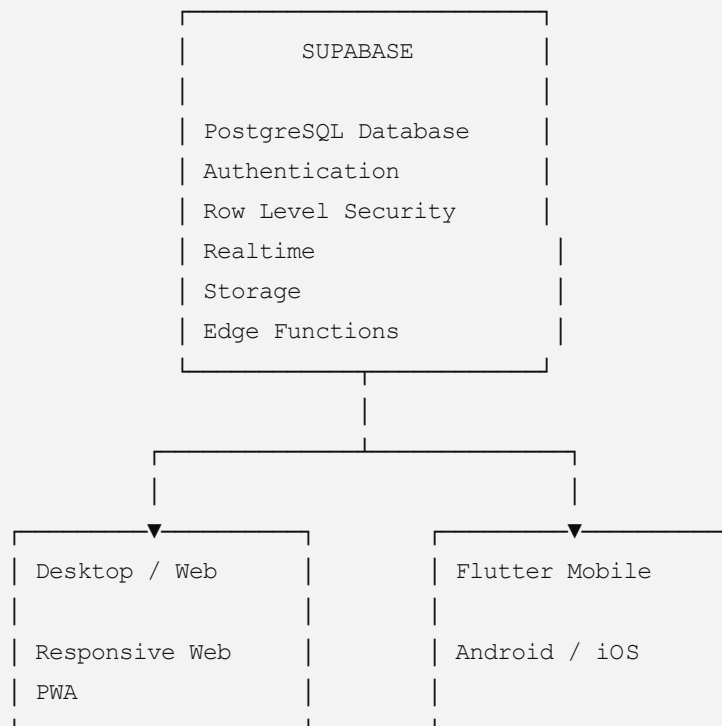
The goal is **not** to simply copy the old Google Sheets system into Supabase.

Instead:

Keep the existing business workflows, but redesign the backend so the system can safely support multiple devices simultaneously.

2. New System Architecture

High-Level Concept



The important principle is:

Desktop and mobile are two clients using the same database.

There should be only **one source of truth for inventory**.

3. Technology Direction

Backend

Use **Supabase** for:

- PostgreSQL database
- Authentication
- Row Level Security

- Realtime synchronization
 - Storage
 - Edge Functions
 - Database functions/triggers where appropriate
-

4. Desktop Application

The desktop application can remain a web application.

Possible technology:

```
Web Application
  ↓
Responsive UI
  ↓
Supabase Client
  ↓
Supabase PostgreSQL
```

It can eventually be installed as a PWA so users can launch it like a normal application.

The desktop version should contain the full operational interface.

5. Mobile Application

Use **Flutter** for the mobile application.

The Flutter application will connect to the same Supabase backend.

Initial mobile functions could include:

```
Login
Stocks
Search
Withdraw
Receive
Sales Orders
Basic History
```

Additional workflows can be added later.

6. Main Application Modules

Suggested navigation:

```
Dashboard
Stocks
Beginning Inventory
Withdraw
Received
Sales Orders
Served
3M Yards
Partial Rolls
Search
Audit
Settings
```

Visibility should depend on the user's permissions.

7. Authentication

The current system uses PIN-based login.

The new system should move authentication to Supabase Auth.

Concept:

```
Supabase Auth
  ↓
Authenticated User
  ↓
User Profile
  ↓
Role / Permissions
  ↓
Allowed Actions
```

Possible initial roles:

```
USER
ADMIN
```

Later, the system can support more detailed permissions.

For example:

```
inventory.view
inventory.withdraw
```

```
inventory.receive
inventory.adjust
inventory.delete
salesorder.create
salesorder.update
reports.view
settings.manage
```

8. Important Security Change

The old PIN system is useful for the existing application, but it should **not** be the primary security model of the new system.

The new application should not rely on:

```
if enteredPin == adminPin
```

inside JavaScript.

Instead:

```
User
  ↓
Supabase Auth
  ↓
Database role/profile
  ↓
RLS
  ↓
Authorized operation
```

Security must be enforced by the backend.

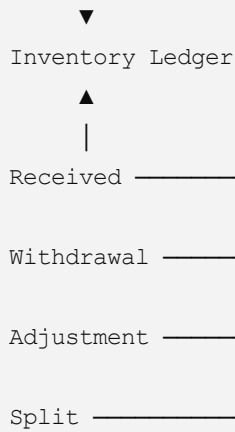
9. Inventory Model

This is one of the most important architectural decisions.

Instead of allowing every screen to directly change a stock quantity, use an **inventory transaction/ledger model**.

Concept:

```
Beginning
|
```



The system should be able to answer:

"Why is the current stock quantity what it is?"

For example:

Beginning	+100
Received	+50
Withdrawal	-20
Adjustment	-5

Current Stock	125

10. Suggested Database Structure

This is a **conceptual schema**, not the final production schema.

profiles

Stores application users.

```
id
email / username
display_name
role
active
created_at
updated_at
```

items

Master item/product information.

```
id
item_code
description
width
width_unit
length
length_unit
category
active
created_at
updated_at
```

Important:

```
item_code
```

should normally be indexed and, if appropriate, unique.

11. Inventory Transactions

Suggested table:

```
inventory_transactions
```

```
id
transaction_date
transaction_type
item_id
quantity
reference_type
reference_id
remarks
created_by
created_at
```

Possible transaction types:

```
BEGINNING
RECEIVED
WITHDRAWAL
SPLIT_PARENT
SPLIT_CHILD
ADJUSTMENT
DISPOSAL
RETURN
```

The exact transaction types should be finalized after the business rules are reviewed.

12. Beginning Inventory

The existing system contains a **Beginning Inventory** workflow.

Conceptually it contains information such as:

```
Date
Item Code
Description
Width
Width Unit
Length
Length Unit
Quantity
Notes
```

New architecture:

```
Beginning Inventory
      ↓
Beginning Record
      ↓
Inventory Transaction
      ↓
Current Stock
```

This means beginning inventory becomes part of the inventory history rather than being a separate number that has to be manually incorporated into stock calculations.

13. Withdrawals

Withdrawal is one of the core workflows.

Existing withdrawal information includes concepts such as:

```
Date
Item Code
Description
Width
Width Unit
Length
Length Unit
Quantity
```

Withdrawal Number
Customer
Remarks
ID

New workflow:

```
graph TD; A[Select Item] --> B[Check Available Stock]; B --> C[Enter Quantity]; C --> D[Validate]; D --> E[Create Withdrawal]; E --> F[Create Inventory Transaction]; F --> G[Stock Updated]; G --> H[Realtime Update];
```

The stock validation should happen at the backend level.

14. Received Inventory

The current system has a **RECEIVED** workflow.

Existing receiving data includes concepts such as:

Date
Item Code
Description
Size
Quantity
MRR Number
Supplier
Remarks
ID
Withdrawal Number / Reference

New workflow:

```
graph TD; A[Receive Item] --> B[Create Receiving Record];
```

↓
Create Inventory Transaction
↓
Increase Stock

15. Roll Splitting

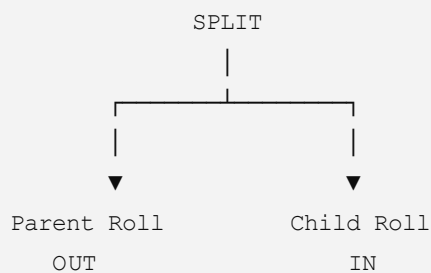
Roll splitting is an important existing workflow.

The existing implementation distinguishes between:

Parent Roll
Child Roll
SPLIT-PARENT
SPLIT-CHILD

The current logic treats the parent roll as an inventory deduction and the child roll as an inventory addition.

Conceptually:



The new system should preserve this concept.

16. Split Transaction

A split should ideally be represented by one common identifier.

Example:

`split_id = SPLIT-000123`

Then:

SPLIT-000123
|
├─ Parent movement

This allows the system to trace the entire operation.

17. Avoid Double Counting Split Rolls

The existing system has an important distinction:

SPLIT-PARENT
SPLIT-CHILD

These are audit/reference records and should not accidentally be counted as additional inventory movements if the actual parent/child movements have already been recorded.

The new system should maintain the same accounting principle.

18. 3M Yards

The existing application has a **3M Yards** workflow.

The source references functionality including:

Add/Edit Roll
Withdraw Yards
Yards Stock

It also uses concepts/tables such as:

YARDSRECEIVED
YARDS-STOCK
YARDSWITHDRAWN

Conceptually:

3M Roll
|
├ Received Yards
|
├ Withdrawn Yards
|
└ Remaining Yards

Possible future tables:

```
yards_rolls  
yards_receipts  
yards_withdrawals
```

The exact business rules should be confirmed before implementation.

19. Partial Rolls

The existing backend has dedicated structures for:

```
PARTIALROLLS  
PARTIALWITHDRAW
```

These should remain a dedicated workflow if they represent a different business process from normal roll inventory.

Possible concept:

```
Full Roll  
  ↓  
Partial Roll  
  ↓  
Partial Stock  
  ↓  
Partial Withdrawal
```

The exact quantity and dimension calculations need to be confirmed from the actual business process.

20. Sales Orders

The existing system contains a **Sales Order** workflow.

Existing data includes concepts such as:

```
ID  
Date  
Time  
Order Number  
Customer  
Remarks  
Items  
Status
```

The workflow contains statuses such as:

```
pending
preparing
ready
done
```

21. Recommended Sales Order Structure

Instead of storing all order items as one large field, use two related tables.

```
sales_orders
```

```
id
order_no
order_date
order_time
customer
remarks
status
created_by
created_at
updated_at
```

```
sales_order_items
```

```
id
sales_order_id
item_id
quantity
unit
notes
```

This makes reporting and inventory integration much easier.

22. Sales Order Workflow

Possible flow:

```
PENDING
  ↓
PREPARING
  ↓
READY
```

↓
DONE

The exact rules for when inventory is deducted should be decided before implementation.

For example:

```
Order Created
  ↓
Does NOT necessarily deduct stock
  ↓
Order Prepared
  ↓
Stock Deducted?
```

This is an important business-rule decision.

23. Served

The existing application also contains a **SERVED** workflow.

The source includes concepts such as:

```
Job Order
Item ID
Width
Length
Quantity
Unit
Customer
Urgency
Status
Sales
Served Timestamp
Row Key
ID
```

This should be preserved in the new system.

The final database relationship between:

```
Sales Order
  ↓
Served
```

should be defined before implementation.

24. Search

The existing system contains search/autocomplete functionality.

The new system should provide centralized search across:

```
Item Code
Description
Order Number
Customer
Withdrawal Reference
Roll
```

Recommended database indexes:

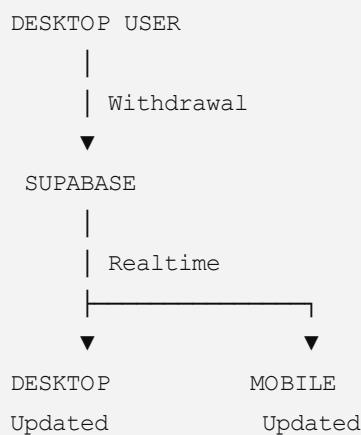
```
items.item_code
items.description
sales_orders.order_no
sales_orders.customer
inventory_transactions.reference_id
```

For larger datasets, PostgreSQL search features can be added later.

25. Realtime Synchronization

This is one of the major benefits of the new architecture.

Example:



If someone withdraws inventory from the desktop:

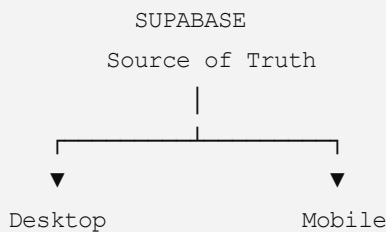
The mobile application should see the updated stock without requiring a separate Google Sheets synchronization process.

26. One Source of Truth

The new architecture should avoid:

```
Desktop Stock Calculation
+
Mobile Stock Calculation
+
Google Sheets Stock Calculation
```

Instead:



Both clients use the same backend rules.

27. Offline Support

The current system already contains concepts related to pending synchronization/local state.

The new mobile application could eventually support:

```
ONLINE
↓
Write to Supabase
```

and:

```
OFFLINE
↓
Save pending operation locally
↓
Reconnect
↓
Sync
↓
Confirm
```

However:

Offline inventory transactions should be Phase 2, not the first implementation.

First make the online transaction system reliable.

28. Audit Trail

The new system should have a proper audit trail.

Suggested:

```
audit_logs
```

```
id
user_id
action
entity_type
entity_id
old_data
new_data
created_at
```

Possible events:

```
CREATE withdrawal
UPDATE withdrawal
VOID withdrawal
CREATE receiving
CREATE beginning inventory
CREATE sales order
UPDATE sales order
CREATE roll split
CHANGE order status
UPDATE item
CHANGE user permission
```

29. Deletion Strategy

The old system uses soft-delete concepts.

The new system should generally avoid physically deleting important inventory history.

Instead:

```
active
deleted_at
deleted_by
```

Or, even better for financial/inventory movements:

```
Original Transaction
    ↓
Reversal / Void Transaction
```

Example:

```
Withdrawal -100
    ↓
Mistake discovered
    ↓
Reversal +100
```

This preserves history.

30. Database Security

Supabase Row Level Security should be enabled.

Conceptually:

```
USER
|
├─ View Stock
├─ Search
├─ Withdraw
├─ Receive
└─ Sales Orders
```

while:

```
ADMIN
|
├─ User Management
├─ Inventory Adjustment
├─ Master Data
├─ Audit
└─ Administrative Actions
```

The exact permission matrix should be defined before production.

31. Transaction Safety

Inventory-changing operations should be atomic.

Example: Roll Split.

```
BEGIN TRANSACTION
```

1. Check parent stock
2. Deduct parent
3. Create child
4. Record split
5. Record audit

```
COMMIT
```

If something fails:

```
ROLLBACK
```

This prevents:

```
Parent deducted  
BUT  
Child not created
```

which would create an incorrect inventory balance.

32. Suggested Database Tables

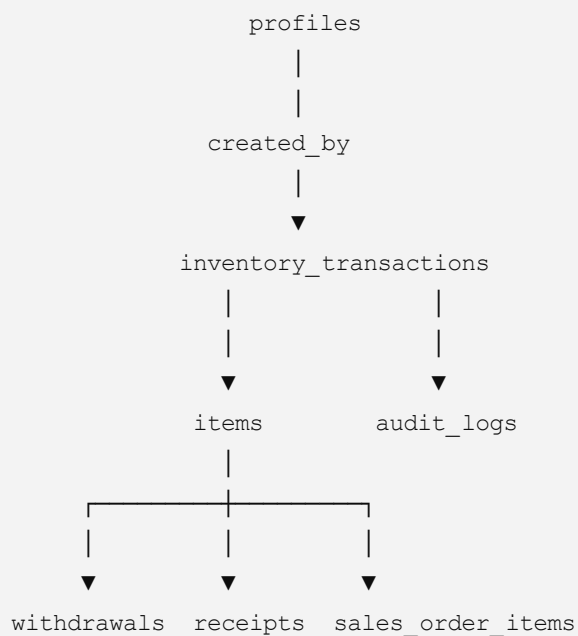
Initial conceptual list:

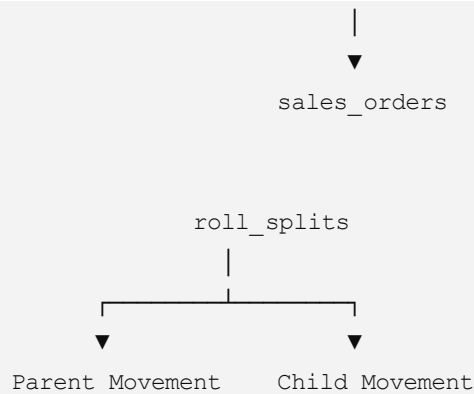
Table	Purpose
profiles	User profile and role
items	Item master
inventory_transactions	Central inventory ledger
beginning_inventory	Beginning inventory
withdrawals	Withdrawal records

Table	Purpose
receipts	Received inventory
sales_orders	Sales order header
sales_order_items	Sales order lines
served_records	Served/job records
roll_splits	Roll split relationships
yards_rolls	3M roll information
yards_receipts	Yards received
yards_withdrawals	Yards withdrawn
partial_rolls	Partial rolls
partial_withdrawals	Partial roll withdrawals
audit_logs	Audit history

This is only the first draft. Some tables may be combined or separated after the business rules are fully understood.

33. Conceptual Database Relationship





34. Inventory Calculation

Conceptually:

CURRENT STOCK

- = Beginning
- + Received
- + Positive Adjustments
- + Split Child
- + Returns
- Withdrawals
- Split Parent
- Disposal
- Negative Adjustments

The final implementation should calculate this consistently from the database.

The desktop and mobile applications should **not each have their own stock formula**.

35. Migration From Google Sheets

The migration should happen in stages.

Phase 1 — Identify Existing Data

Review the current Sheets/tabs and map their meaning.

The existing backend references concepts including:

- ITEMCODE
- CODEMAP
- WITHDRAWAL
- RECEIVED
- YARDSRECEIVED

YARDS-STOCK
YARDSWITHDRAWN
PARTIALROLLS
PARTIALWITHDRAW
DISPOSAL
BEGINNING
SALESORDER
SPLIT
SPLIT-PARENT
SPLIT-CHILD
SERVED

These should be verified against the actual production spreadsheet before migration.

36. Data Cleanup

Before importing:

```
Normalize item codes
      ↓
Check duplicates
      ↓
Check missing references
      ↓
Check quantities
      ↓
Normalize dates
      ↓
Normalize units
      ↓
Check deleted records
      ↓
Check split relationships
```

37. Migration Order

Recommended:

First

Items
Users
Reference Data

Second

Beginning Inventory
Received
Withdrawals

Third

Roll Splits
Partial Rolls
3M Yards

Fourth

Sales Orders
Served

Finally

Audit / Historical Records

38. Stock Reconciliation

Before switching completely to Supabase:

Google Sheets Stock
VS
Supabase Stock

The values should be compared.

For each important item:

Old System Balance
=
New System Balance

Any differences should be investigated before going live.

39. Development Phases

Phase 1 — Foundation

Build:

Supabase
Authentication
Profiles
Roles
Items
RLS
Database migrations

Phase 2 — Inventory Core

Build:

Stocks
Beginning Inventory
Received
Withdrawals
Inventory Ledger
Stock Calculation

Goal:

Make the core inventory system correct before adding complex workflows.

Phase 3 — Sales Orders

Build:

Sales Orders
Order Items
Status Workflow
Customer
Preparation
Served

Phase 4 — Roll Operations

Build:

Roll Splitting
Parent Roll

Child Roll
Split History
Audit

Phase 5 — Special Inventory

Build:

3M Yards
Partial Rolls
Yards Withdrawal
Disposal

Phase 6 — Search and Reporting

Build:

Global Search
Filters
Inventory History
Audit Reports
Stock Reports
Operational Reports

Phase 7 — Flutter Mobile

Start with:

Login
Stocks
Search
Withdraw
Receive
Sales Orders
History

Then add advanced features.

Phase 8 — Realtime / Offline

Add:

Realtime subscriptions
Local caching
Offline queue
Retry handling
Conflict handling

40. Suggested Web Project Structure

```
web/  
|  
├─ src/  
|   |  
|   ├─ components/  
|   |  
|   ├─ pages/  
|   |  
|   ├─ features/  
|   |   |  
|   |   ├─ stocks/  
|   |   |  
|   |   ├─ withdrawals/  
|   |   |  
|   |   ├─ receiving/  
|   |   |  
|   |   ├─ sales-orders/  
|   |   |  
|   |   ├─ roll-splits/  
|   |   |  
|   |   ├─ yards/  
|   |   |  
|   |   ├─ partial-rolls/  
|   |   |  
|   |   └─ audit/  
|   |  
|   ├─ services/  
|   |   |  
|   |   ├─ supabase/  
|   |   |  
|   |   └─ inventory/  
|   |  
|   └─ state/  
|   |  
|   └─ utils/  
|   |  
└─ ...
```

41. Suggested Flutter Structure

```
mobile/  
|  
└─ lib/  
    |  
    ├─ core/  
    |  
    └─ auth/
```

```
├─ stocks/
├─ withdrawals/
├─ receiving/
├─ sales_orders/
├─ roll_splits/
├─ yards/
├─ partial_rolls/
├─ audit/
└─ shared/
```

42. Supabase Structure

```
supabase/
|
├─ migrations/
|
├─ functions/
|   ├─ inventory/
|   ├─ roll_split/
|   └─ reports/
|
└─ seed/
```

43. What Should NOT Be Copied From the Old System

The existing system should be treated as the **business-process reference**, not as the technical architecture to copy.

Avoid carrying forward:

Google Sheets as the primary database

Apps Script CRUD architecture

Client-side security

Browser-only authorization

Duplicated stock calculations

Large spreadsheet-dependent data structures

Manual synchronization as the normal workflow

Instead:

```
Existing Business Logic
      ↓
Redesigned Database Model
      ↓
Supabase
      ↓
Desktop + Mobile
```

44. Important Business Questions

Before building the final schema, these questions need to be answered.

Items

1. Is every item code unique?
2. Can one item code have multiple sizes?
3. Are width and length always numeric?
4. What units are allowed?
5. What item categories exist?

Inventory

6. Can quantity contain decimals?
7. Is negative stock allowed?
8. Who can perform inventory adjustments?
9. How are stock corrections handled?
10. Is disposal an inventory transaction?

Withdrawals

11. What exactly is the withdrawal number?
12. Can one withdrawal contain multiple items?
13. Can a withdrawal be cancelled?
14. When exactly does withdrawal affect stock?

Receiving

15. Is MRR mandatory?

16. Can one MRR contain multiple items?

17. How are receiving errors corrected?

Roll Splitting

18. Can one parent create multiple child rolls?

19. Can the child quantity differ from the parent?

20. How is the child quantity calculated?

21. Can a split be reversed?

22. Can a partial roll be split again?

3M Yards

23. What exactly defines a 3M roll?

24. How are yards calculated?

25. Can one roll have multiple withdrawals?

26. Can a yard withdrawal be reversed?

Partial Rolls

27. What exactly qualifies as a partial roll?

28. How is remaining quantity calculated?

29. Can partial rolls themselves be split?

Sales Orders

30. Does creating an order reserve stock?

31. At what point is stock deducted?

32. Can orders be cancelled?

33. Can one order contain multiple items?

34. What is the exact relationship between Sales Order and Served?

Users

35. Are USER and ADMIN enough?

36. Should permissions be module-based?

37. Who can adjust inventory?

38. Who can void records?

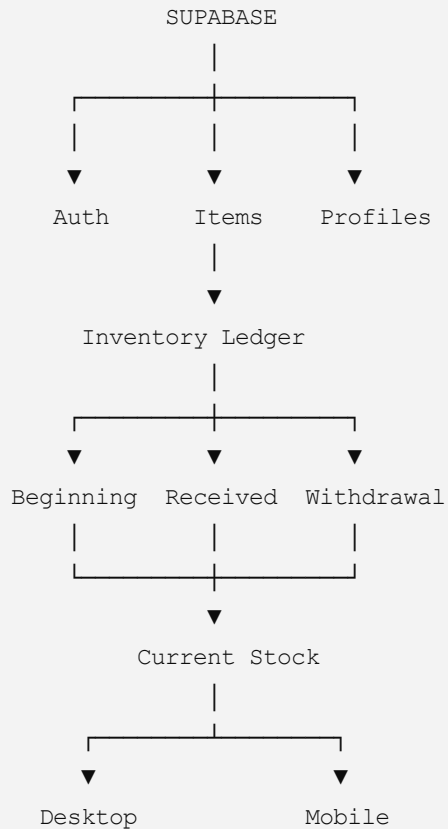
39. Who can access reports?

40. Who can manage users?

45. Recommended First Milestone

Do **not** rebuild every existing screen immediately.

First build this:



Then prove that:

Beginning
+ Received
- Withdrawal
= Correct Stock

and that the result is identical on:

Desktop A
Desktop B
Mobile

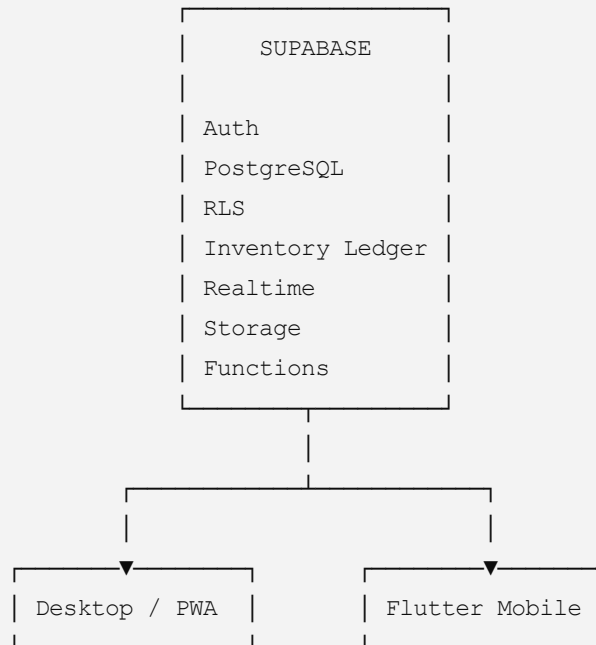
Once that foundation works, add:

Sales Orders
Served
Roll Splitting
3M Yards
Partial Rolls
Disposal
Reports

46. Final Concept

The new YUNITRADE system should be treated as a **shared inventory platform**, rather than simply a Google Sheets replacement.

The target architecture is:



Core principle

ONE DATABASE. ONE INVENTORY TRUTH. MULTIPLE CLIENTS.

The existing YUNITRADE system provides the business workflow.

Supabase provides the new backend foundation.

The desktop web application and Flutter mobile application become two interfaces to the same inventory system.

Next step

I would recommend that the **next document we create is the actual Supabase database schema**, starting with:

```
profiles
items
inventory_transactions
beginning_inventory
withdrawals
receipts
```

```
sales_orders  
sales_order_items  
roll_splits  
audit_logs
```

Then we can map the existing Google Sheet columns from `Code.gs` into those tables **before writing any new frontend code.**